

# Manual Práctico Definitivo: React Native

Cómo traducir los enunciados de examen a código (Paso a Paso)

Preparación para el 10

## Índice

|          |                                                                                  |           |
|----------|----------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introducción: Cómo leer el examen</b>                                         | <b>2</b>  |
| <b>2</b> | <b>Receta 1: El Listado (Mostrar datos de la BD en pantalla)</b>                 | <b>3</b>  |
| 2.1      | Paso 1: Importar lo necesario . . . . .                                          | 3         |
| 2.2      | Paso 2: Crear el estado (Justo debajo de 'export default function...') . . . . . | 3         |
| 2.3      | Paso 3: El useEffect (Pedir los datos) . . . . .                                 | 3         |
| 2.4      | Paso 4: El renderItem (Cómo se ve cada fila) . . . . .                           | 3         |
| 2.5      | Paso 5: El Return (Pintar el FlatList) . . . . .                                 | 4         |
| <b>3</b> | <b>Receta 2: Formularios (Crear o Editar con Formik)</b>                         | <b>5</b>  |
| 3.1      | Diferencia clave entre Crear y Editar . . . . .                                  | 5         |
| 3.2      | Paso 1: Preparar los Valores Iniciales y la Validación . . . . .                 | 5         |
| 3.3      | Paso 2: (SÓLO SI ES EDITAR) Cargar los datos actuales . . . . .                  | 5         |
| 3.4      | Paso 3: El onSubmit (La función que guarda los datos) . . . . .                  | 5         |
| 3.5      | Paso 4: El return con Formik . . . . .                                           | 6         |
| 3.6      | La gran trampa de los exámenes: El DropDownPicker . . . . .                      | 6         |
| <b>4</b> | <b>Receta 3: Borrar con Modal (DELETE)</b>                                       | <b>8</b>  |
| 4.1      | Paso 1: Estado para controlar el modal . . . . .                                 | 8         |
| 4.2      | Paso 2: La función de borrado real . . . . .                                     | 8         |
| 4.3      | Paso 3: El botón en la fila de la lista . . . . .                                | 8         |
| 4.4      | Paso 4: El componente DeleteModal (MUY IMPORTANTE) . . . . .                     | 8         |
| <b>5</b> | <b>Receta 4: Acciones Rápidas (Cambios de estado con 1 clic)</b>                 | <b>10</b> |
| 5.1      | Paso 1: La función de cambio . . . . .                                           | 10        |
| 5.2      | Paso 2: El botón y sus "Trampas" . . . . .                                       | 10        |
| <b>6</b> | <b>Receta 5: Analíticas y Cabeceras en Listas</b>                                | <b>11</b> |
| 6.1      | Paso 1: Estado y Fetch para las analíticas . . . . .                             | 11        |
| 6.2      | Paso 2: Definir el renderHeader . . . . .                                        | 11        |
| 6.3      | Paso 3: Pasárselo al FlatList . . . . .                                          | 11        |
| <b>7</b> | <b>El "Cheat Sheet" del Navegador</b>                                            | <b>12</b> |

# 1 Introducción: Cómo leer el examen

En los exámenes de DeliverUS, los profesores nunca te pedirán que programes algo desde cero sin darte pistas. El secreto está en saber identificar la **\*\*Acción Clave\*\*** en el enunciado.

Cada vez que leas el examen, busca estas frases y aplica la "receta" correspondiente que verás en las siguientes secciones:

| Si el enunciado dice...                       | Tienes que programar...                                                                                                    |
|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <i>"ver un listado con todos los..."</i>      | <b>Receta 1:</b> Un estado ( <code>useState</code> ), un <code>useEffect</code> que haga GET, y un <code>FlatList</code> . |
| <i>"crear un nuevo..." o "editar un..."</i>   | <b>Receta 2:</b> Formulario con <code>Formik</code> , validación de <code>Yup</code> , y petición POST o PUT.              |
| <i>"asignar un horario a..."</i>              | <b>Receta 2 (Variante):</b> Formulario con un <code>DropDownPicker</code> .                                                |
| <i>"borrar / eliminar..."</i>                 | <b>Receta 3:</b> <code>DeleteModal</code> , un estado para guardar el ID a borrar, y un DELETE.                            |
| <i>"cambiar de estado" o "marcar como..."</i> | <b>Receta 4:</b> Un simple <code>Pressable</code> que llame a una función PATCH/PUT y recargue la lista.                   |
| <i>"visualizar analíticas" o "cabecera"</i>   | <b>Receta 5:</b> Usar el <code>ListHeaderComponent</code> dentro del <code>FlatList</code> .                               |

A partir de aquí, vamos a desglosar exactamente **qué líneas de código escribir** para cada receta.

## 2 Receta 1: El Listado (Mostrar datos de la BD en pantalla)

**Cuándo usarla:** Examen Direcciones (RF1), Pedidos (RF1), Horarios (RF2), Categorías (RF1).

El enunciado siempre te dirá: *"Mostrar en pantalla todos los [pedidos/direcciones/horarios]"*.

### 2.1 Paso 1: Importar lo necesario

Normalmente el archivo ya existe, pero asegúrate de tener:

```
import React, { useState, useEffect } from 'react'
import { FlatList, View, Text, Pressable } from 'react-native'
import { showMessage } from 'react-native-flash-message'
// Importa la funcion que te de el profesor para hacer el GET (miralo en el enunciado)
import { getEndpoints } from '../api/ElArchivoQueSeaEndpoints'
```

### 2.2 Paso 2: Crear el estado (Justo debajo de 'export default function...')

Necesitamos una variable donde guardar la lista que nos dé el servidor.

```
export default function PantallaDelExamen({ navigation, route }) {
  // 1. Estado inicializado como un array vacio
  const [items, setItems] = useState([])
```

### 2.3 Paso 3: El useEffect (Pedir los datos)

Este código va justo debajo del useState. Es el encargado de llamar al servidor.

```
useEffect(() => {
  async function fetchItems() {
    try {
      // Llama a la funcion que te dice el enunciado, pasandole IDs si hace falta
      const fetchedItems = await getEndpoints(route.params.id)
      setItems(fetchedItems) // Guardamos los datos en el estado
    } catch (error) {
      showMessage({ message: 'Error al cargar los datos', type: 'error' })
    }
  }
  fetchItems() // Ejecutamos la funcion
}, [route]) // VITAL: Poner 'route' aqui hace que la lista se refresque al volver de otra pantalla.
```

### 2.4 Paso 4: El renderItem (Cómo se ve cada fila)

¿Cómo quiere el profesor que se vea la fila? Fíjate en la captura del examen. Si te pide que muestres el nombre y un dato más, hazlo así:

```
const renderItem = ({ item }) => {
  return (
    <View style={styles.card}>
      <Text>{item.name}</Text>

      {/* Caso Examen Horarios: Mostrar horario si existe */}
      {item.schedule ? (
        <Text>{item.schedule.startTime} - {item.schedule.endTime}</Text>
      ) : (
        <Text>Not Scheduled</Text> // Si dice "mostrar Not Scheduled si no tiene", es un IF.
      )}
    </View>
  )
}
```

```
    </View>
  )
}
```

## 2.5 Paso 5: El Return (Pintar el FlatList)

```
return (
  <FlatList
    data={items} // Le pasamos el estado que creamos en el Paso 2
    renderItem={renderItem} // Le pasamos la funcion del Paso 4
    keyExtractor={item => item.id.toString()} // Siempre es asi
  />
)
} // Cierre del componente
```

## 3 Receta 2: Formularios (Crear o Editar con Formik)

**Cuándo usarla:** Examen Direcciones (RF2), Categorías (RF2), Horarios (RF4, RF5), Pedidos (RF2).

### 3.1 Diferencia clave entre Crear y Editar

- **Crear:** Haces un POST. Los campos empiezan vacíos.
- **Editar:** Haces un PUT. **OBLIGATORIO:** Al abrir la pantalla hay que hacer un GET para rellenar los datos actuales, y en Formik debes usar `enableReinitialize={true}`.

### 3.2 Paso 1: Preparar los Valores Iniciales y la Validación

Justo dentro del componente:

```
export default function FormScreen({ navigation, route }) {  
  // 1. Estado para los valores del formulario  
  const [initialValues, setInitialValues] = useState({  
    campo1: '',  
    campo2: null  
  })  
  
  // 2. Esquema de validacion (Yup) - Fijaos en el enunciado (ej. "precio obligatorio y mayor que 0")  
  const validationSchema = yup.object().shape({  
    campo1: yup.string().required('El campo 1 es obligatorio'),  
    campo2: yup.number().positive('Debe ser mayor que 0').required()  
  })  
}
```

### 3.3 Paso 2: (SÓLO SI ES EDITAR) Cargar los datos actuales

```
useEffect(() => {  
  async function fetchDetail() {  
    try {  
      const itemDetail = await getDetalleApi(route.params.id)  
      // Sobrescribimos initialValues con los datos que nos ha dado el backend  
      setInitialValues({  
        campo1: itemDetail.campo1,  
        campo2: itemDetail.campo2  
      })  
    } catch (error) {  
      showMessage({ message: 'Error cargando detalle', type: 'error' })  
    }  
  }  
  fetchDetail()  
}, []) // Aquí el array va vacío []
```

### 3.4 Paso 3: El onSubmit (La función que guarda los datos)

```
const guardarDatos = async (values) => {  
  try {  
    // Si es crear: await postApi(values)  
    // Si es editar: await putApi(route.params.id, values)  
    await updateApi(route.params.id, values)  
  
    showMessage({ message: 'Guardado correctamente', type: 'success' })  
    // ESTO ES DE MATRICULA: Volver hacia atrás diciendo que se ha modificado algo (dirty)  
    navigation.navigate('LaPantallaDeLaLista', { dirty: true })  
  }  
}
```

```

    } catch (error) {
      showMessage({ message: 'Error al guardar', type: 'error' })
    }
  }
}

```

### 3.5 Paso 4: El return con Formik

```

return (
  <Formik
    enableReinitialize={true} // <-- OBLIGATORIO SI ES EDICION O NO FUNCIONARA
    initialValues={initialValues}
    validationSchema={validationSchema}
    onSubmit={guardarDatos}
  >
    {({ setFieldValue, values, handleSubmit }) => (
      <ScrollView>
        {/* Inputs normales (Texto) */}
        <InputItem name="campo1" label="Nombre del Campo" />

        {/* BOTON DE GUARDAR */}
        <Pressable onPress={handleSubmit}>
          <Text>Guardar</Text>
        </Pressable>
      </ScrollView>
    )}
  </Formik>
)

```

### 3.6 La gran trampa de los exámenes: El DropdownPicker

A menudo te piden asignar algo (Ej. Asignar Horario a Producto, Asignar Categoría a Producto). Esto no se hace con un InputItem, se hace con un DropdownPicker.

Si en el examen te piden esto, tienes que añadir esto a tu código:

```

// 1. Estados necesarios para el Dropdown (PONER ARRIBA)
const [open, setOpen] = useState(false)
const [opciones, setOpciones] = useState([]) // Aqui guardaremos las opciones del selector

// 2. Hay que hacer un GET de las opciones en el useEffect (Ej. Cargar las categorias)
useEffect(() => {
  async function fetchOpciones() {
    const data = await getOpcionesDelBackend(restaurantId)

    // EL DROPDOWN NECESITA QUE LOS OBJETOS TENGAN 'label' Y 'value'. HAY QUE MAPEARLOS
    .
    let opcionesMapeadas = data.map(item => ({
      label: item.name, // Lo que el usuario lee
      value: item.id    // El ID que se mandara a la base de datos
    }))

    // Si el enunciado dice: "Se puede establecer que no haya horario", anadimos una
    opcion NULA:
    opcionesMapeadas.unshift({ label: 'Sin asignar (Not scheduled)', value: null })

    setOpciones(opcionesMapeadas)
  }
  fetchOpciones()
}, [])

```

```
// 3. Dentro de <Formik> ({ setFieldValue, values ... , metemos el componente:
<DropDownPicker
  open={open}
  setOpen={setOpen}
  value={values.miCampoId} // <-- El nombre de la variable que espera el backend
  items={opciones}
  setItems={setOpciones}
  onSelectItem={item => setFieldValue('miCampoId', item.value)} // <-- CRUCIAL:
  Actualizar formik
/>
```

## 4 Receta 3: Borrar con Modal (DELETE)

**Cuándo usarla:** Direcciones (RF4), Categorías (RF3), Horarios (RF3).

Si el enunciado dice *"solicitar confirmación antes del borrado"* o *"usando un componente Modal"*, aplica esto EXACTAMENTE.

### 4.1 Paso 1: Estado para controlar el modal

En la pantalla donde tienes el listado (FlatList):

```
// Guardara el objeto entero que el usuario quiere borrar (empieza siendo null)
const [itemToBeDeleted, setItemToBeDeleted] = useState(null)
```

### 4.2 Paso 2: La función de borrado real

```
const removeElement = async (item) => {
  try {
    // 1. Llamada a la API (pasandole el ID del objeto)
    await deleteApiFunction(item.id)
    // 2. Refrescar la lista de la pantalla (Llamar a la misma funcion que tienes en tu
    useEffect)
    await fetchItems()
    // 3. Cerrar el modal
    setItemToBeDeleted(null)
    // 4. Avisar al usuario
    showMessage({ message: 'Borrado con exito', type: 'success' })
  } catch (error) {
    setItemToBeDeleted(null) // Cerramos modal aunque falle
    showMessage({ message: 'Error al borrar', type: 'error' })
  }
}
```

### 4.3 Paso 3: El botón en la fila de la lista

Dentro de tu renderItem:

```
<Pressable onPress={() => setItemToBeDeleted(item)}>
  <MaterialCommunityIcons name='delete' size={20} color='white' />
</Pressable>
```

### 4.4 Paso 4: El componente DeleteModal (MUY IMPORTANTE)

Este componente **\*\*NUNCA\*\*** va dentro del renderItem ni dentro del FlatList. Va al final de tu componente, justo antes de cerrar la vista principal.

```
return (
  <> { /* Fragmento para poder devolver el FlatList y el Modal a la vez */}
    <FlatList
      data={items}
      renderItem={renderItem}
      keyExtractor={item => item.id.toString()}
    />

    { /* EL MODAL SE COLOCA AQUI ABAJO */}
    <DeleteModal
      isVisible={itemToBeDeleted !== null} // Se muestra si itemToBeDeleted no es nulo
      onCancel={() => setItemToBeDeleted(null)} // Si pulsa cancelar, reseteamos el
      estado
    />
  </>
)
```



```
      onConfirm={() => removeElement(itemToBeDeleted)} // Si confirma, llamamos a la
funcion
    >
      <Text>¿Seguro que deseas borrar este elemento?</Text>
    </DeleteModal>
  </>
)
```

## 5 Receta 4: Acciones Rápidas (Cambios de estado con 1 clic)

**Cuándo usarla:** Direcciones (RF3: Marcar como predeterminada), Pedidos (RF4: Avanzar de estado).

El enunciado pide cambiar un estado sin formularios ni confirmaciones. Pulsas un botón y ocurre la magia.

### 5.1 Paso 1: La función de cambio

```
const cambiarEstadoRapido = async (item) => {
  try {
    // Hacemos el PATCH o PUT a la API (la funcion te la dan hecha en el fichero de Endpoints)
    await accionRapidaApi(item.id)

    // CRUCIAL: Como en el borrado, hay que volver a pedir los datos para que se vea el cambio
    await fetchItems()

    showMessage({ message: 'Operacion realizada', type: 'success' })
  } catch (error) {
    showMessage({ message: 'No se pudo realizar la operacion', type: 'error' })
  }
}
```

### 5.2 Paso 2: El botón y sus "Trampas"

Añadimos un Pressable en nuestro renderItem.

**Trampa típica de examen:** A veces el enunciado dice *"No se puede avanzar si el estado ya es 'delivered'"*. Tienes que esconder el botón.

```
{/* Renderizado condicional: Solo mostramos el boton si NO esta entregado */}
{item.status !== 'delivered' && (
  <Pressable onPress={() => cambiarEstadoRapido(item)}>
    <Text>Avanzar estado</Text>
  </Pressable>
)}
```

**Otra trampa (Direcciones):** *"Pintar una estrella si es predeterminada"*.

```
<Pressable onPress={() => cambiarEstadoRapido(item)}>
  {/* Cambiamos el color condicionalmente */}
  <MaterialCommunityIcons
    name='star'
    color={item.isDefault ? 'gold' : 'grey'}
  />
</Pressable>
```

## 6 Receta 5: Analíticas y Cabeceras en Listas

**Cuándo usarla:** Examen Pedidos (RF3: Analíticas).

Cuando te piden mostrar estadísticas u otra información **arriba del todo** de una lista de elementos (y que esa info suba al hacer scroll), debes usar `ListHeaderComponent`.

### 6.1 Paso 1: Estado y Fetch para las analíticas

Si las analíticas vienen en un objeto separado, hazle su estado y su petición.

```
const [analytics, setAnalytics] = useState(null)

// Dentro de tu useEffect donde cargas la lista, cargas tambien esto:
const fetchedAnalytics = await getAnalyticsApi()
setAnalytics(fetchedAnalytics)
```

### 6.2 Paso 2: Definir el renderHeader

Fuera del return, defines cómo se verá la cabecera.

```
const renderHeader = () => {
  return (
    // Te suelen dar los estilos creados. Usalos.
    <View style={styles.analyticsContainer}>
      <Text>Ingresado Hoy: {analytics?.invoicedToday} E</Text>
      <Text>Pedidos Pendientes: {analytics?.numPendingOrders}</Text>
    </View>
  )
}
```

### 6.3 Paso 3: Pasárselo al FlatList

```
return (
  <FlatList
    data={pedidos}
    renderItem={renderPedido}
    ListHeaderComponent={renderHeader} // ESTO ES LA MAGIA
  />
)
```

## 7 El "Cheat Sheet" del Navegador

Para terminar, cómo moverte entre pantallas y pasar información (como el ID del restaurante).

### Para enviar a otra pantalla (Desde el origen):

```
<Pressable
  onPress={() => navigation.navigate('NombreDeLaPantallaDestino', {
    id: restaurante.id,    // Pasas el ID del restaurante
    productoId: item.id    // O el ID del producto
  })}
>
  <Text>Ir a la otra pantalla</Text>
</Pressable>
```

### Para leer esos datos (En el destino):

```
export default function PantallaDestino({ navigation, route }) {
  // Sacamos las variables de route.params
  const elIdDelRestaurante = route.params.id
  const elIdDelProducto = route.params.productoId

  // Y ya las podemos usar en nuestro useEffect para hacer peticiones...
}
```